

PEMANFAATAN ALGORITMA GREEDY DALAM PEMBUATAN *BOT* PERMAINAN ROBOCODE TANK ROYALE

TUGAS BESAR

Diajukan sebagai syarat menyelesaikan Mata Kuliah Strategi Algoritma (IF25-21013) Kelas
RA

Di Program Studi Teknik Informatika, Fakultas Teknologi Industri, Institut Teknologi
Sumatera



OLEH: TANK Q

Valentino Glen Lambok Munthe 124140022

Roy Kurniawan 124140024

Nazhifa Dwi Ralya 124140042

Dosen Pengampu: Imam Eko Wicaksono, S.Si., M.Si.

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS TEKNOLOGI INDUSTRI
INSTITUT TEKNOLOGI SUMATERA
2026**

DAFTAR ISI

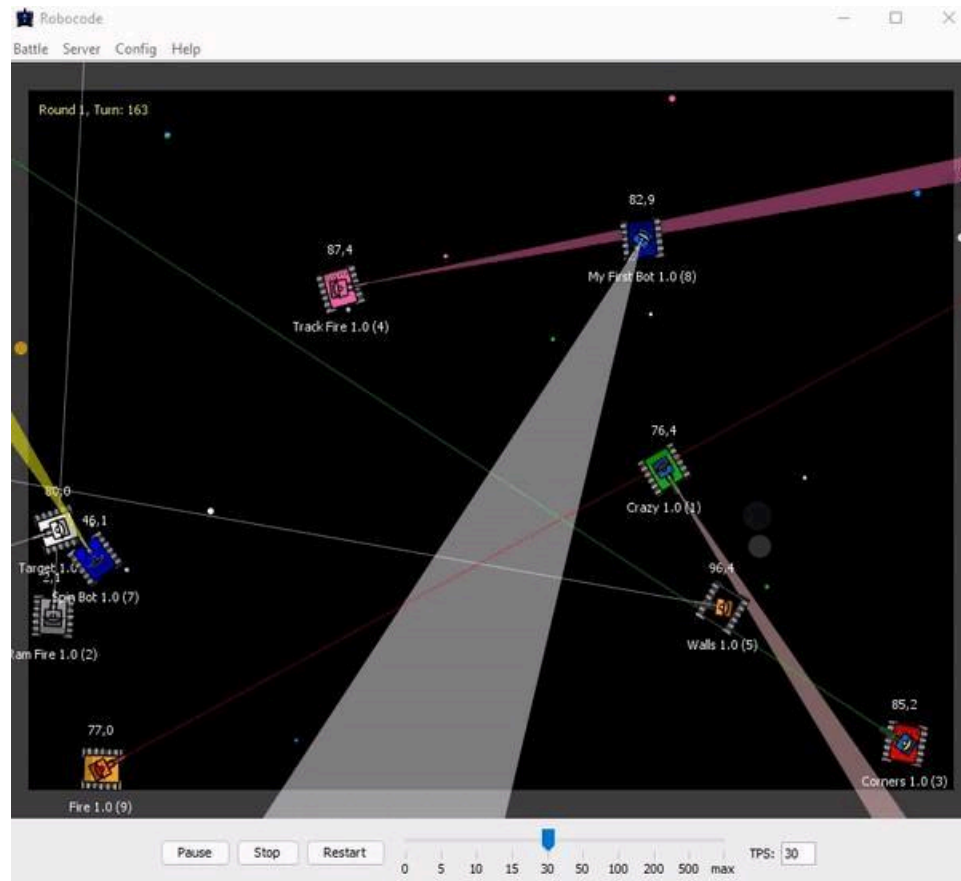
DAFTAR ISI	1
BAB I	3
DESKRIPSI TUGAS	3
BAB II	
LANDASAN TEORI	10
2.1 Dasar Teori	10
2.2 Cara Kerja Program Secara Umum	11
2.3 Implementasi Algoritma Greedy pada Bot	12
2.3.1 Pemilihan Target Musuh	12
2.3.2 Penentuan Arah Gerak	12
2.3.3 Sistem Penghindaran Serangan	12
2.3.4 Sistem Penembakan	13
2.3.5 Cara Menjalankan Bot	13
BAB III	14
APLIKASI STRATEGI GREEDY	14
3.1 Mapping Persoalan Robocode Tank Royale ke Elemen-Elemen Algoritma Greedy	14
3.2 Eksplorasi Alternatif Solusi Greedy dalam persoalan Robocode Tank Royale	15
3.2.1 Alternatif 1 - TANK Q	15
3.2.2 Alternatif 2 - BotKill	15
Heuristic	15
3.2.3 Alternatif 3 - BotViper	15
Heuristic	15
3.2.4 Alternatif 4 - Velora	15
Heuristic	15
3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi Greedy	16
3.4 Strategi Greedy yang Dipilih	17
BAB IV	18
IMPLEMENTASI DAN PENGUJIAN	18
4.1 Implementasi Keempat Alternatif Solusi	18
4.1.1 Pseudocode Alternatif 1: TANK Q	18
4.1.2 Pseudocode Alternatif 2: BotKill	24
4.1.3 Pseudocode Alternatif 3: BotViper	26
4.1.4 Pseudocode Alternatif 4: Velora	28
4.2 Penjelasan Fungsi, Struktur Data, dan Prosedur	31
4.2.1 Struktur Data	31
4.2.2 Fungsi Utama	32
4.2.3 Prosedur	35
4.3 Pengujian	35

4.3.1 Hasil Pengujian: BotKill	36
4.3.2 Hasil Pengujian: BotViper	37
4.3.3 Hasil Pengujian: VELORA	37
4.3.4 Hasil Pengujian Mengalami Kejadian Unik	37
4.4 Analisis Hasil Pengujian	38
BAB 5	39
KESIMPULAN DAN SARAN	39
5.1 Kesimpulan	39
5.2 Saran	39
LAMPIRAN	41
DAFTAR PUSTAKA	42

BAB I

DESKRIPSI TUGAS

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran.



Gambar 1. Robocode Tank Royale

Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1. Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

2. Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

3. Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.

- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

4. Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

5. Panas Meriam (Gun Heat)

Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

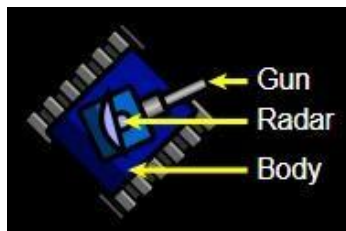
6. Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain.

Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

7. Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Body adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.

Gun digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.

Radar digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

8. Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

9. Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

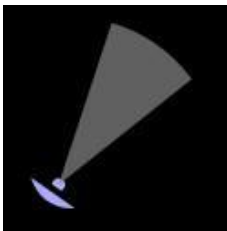
Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

10. Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar. Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh.

11. Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perangkingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi.

Untuk informasi lebih lengkap, silahkan buka dokumentasi Tank Royale pada link [berikut](#).

Starter Pack

Untuk tugas besar ini, game engine yang akan digunakan sudah dimodifikasi oleh asisten. Berikut adalah beberapa perubahan yang dibuat oleh asisten dari game engine Tank Royale:

- **Theme GUI:** diubah menjadi light theme

- **Skor & Energi:** masing-masing bot ditampilkan di samping arena permainan agar lebih mudah diamati saat pertarungan
- **Turn Limit:** Durasi pertarungan tidak akan bergantung pada banyak ronde. Pada game engine ini, pertarungan akan berakhir apabila banyaknya turn sudah mencapai batas tertentu. Apabila batasan ini tercapai, ronde otomatis langsung berakhir dan pemenang pertarungan akan ditampilkan. Turn Limit dapat diatur pada menu “setup rules”. Source Code untuk game engine dan template bot telah disediakan pada tautan berikut.

[tubes1-if2211-starter-pack](#)

Adapun panduan mengenai cara menjalankan game engine, membuat bot, dan melihat referensi API dapat dilihat melalui tautan berikut: [Starter Guide](#).

Isi laporan

- **Cover:** Cover laporan ada foto anggota kelompok (foto bertiga).
- **Bab 1:** Deskripsi tugas (dapat menyalin spesifikasi tugas ini).
- **Bab 2:** Landasan Teori.
 - 0 Dasar teori (algoritma *greedy*) secara umum
 - o Bagaimana cara kerja program secara umum (bagaimana bot melakukan aksinya, bagaimana mengimplementasikan algoritma *greedy* ke dalam bot, bagaimana menjalankan bot, dll).
- **Bab 3:** Aplikasi strategi *greedy*.
 - 0 Proses mapping persoalan Robocode Tank Royale menjadi elemen-elemen algoritma Greedy (himpunan kandidat, himpunan solusi, fungsi solusi, fungsi seleksi, fungsi kelayakan, fungsi objektif)
 - o Eksplorasi **4 alternatif solusi** greedy yang mungkin dipilih dalam persoalan Robocode Tank Royale. (setiap alternatif harus memiliki **heuristic** yang **berbeda**)
 - o Analisis efisiensi dan efektivitas dari kumpulan alternatif solusi greedy yang dirumuskan
 - o Strategi *greedy* **yang dipilih** dari 4 alternatif solusi greedy (yang akan diimplementasikan dalam program) beserta alasan dan pertimbangan pemilihan strategi tersebut.
- **Bab 4:** Implementasi dan pengujian.
 - 0 Implementasi 4 alternatif solusi (pseudocode yang cukup detail sehingga **dapat dimengerti dengan mudah**. Berikan komen pada pseudocode jika merasa perlu)
 - o Penjelasan struktur data, fungsi, dan prosedur yang digunakan pada bot dengan solusi greedy **yang dipilih**.
 - o Pengujian dilakukan minimal sebanyak 3 kali berupa pertandingan antara bot utama (**yang dipilih**) dengan **bot asisten** (format 1 vs 1). Pengujian yang dicantumkan sebaiknya mencakup semua kejadian yang unik.
 - o Analisis hasil dari pengujian. Misalnya adalah apakah strategi *greedy* berhasil mendapatkan nilai optimal, lalu jika tidak, dalam kondisi seperti apa strategi *greedy* tidak berhasil mendapatkan nilai optimal, dsb.
- **Bab 5:** Kesimpulan dan saran.

- **Lampiran:** Tautan *repository* GitHub dan video (jika membuat)
- **Daftar Pustaka**

Keterangan laporan:

1. Laporan ditulis dalam bahasa Indonesia yang baik dan benar.
2. Laporan mengikuti format pada *section* “Isi laporan” dengan baik dan benar.
3. Identitas per halaman harus jelas (misalnya : halaman, kode kuliah).

BAB II

LANDASAN TEORI

2.1 Dasar Teori

Algoritma Greedy adalah algoritma yang memecahkan persoalan optimasi secara langkah demi langkah (*step-by-step*). Pada setiap langkahnya, algoritma ini mengambil keputusan berdasarkan dua prinsip utama:

1. Mengambil pilihan terbaik yang dapat diperoleh pada saat itu tanpa memperhatikan konsekuensi ke depan (prinsip "*take what you can get now*" atau optimum lokal).
2. Berharap bahwa rangkaian dari pilihan optimum lokal pada setiap langkah tersebut akan menghasilkan solusi akhir yang optimum secara keseluruhan (optimum global)

Dalam menjalankan **Algoritma Greedy**, terdapat beberapa elemen penting yang mempermudah proses penyelesaian masalah, yaitu :

1. **Himpunan Kandidat (C)** : berisi elemen yang dapat dipilih pada langkah, seperti simpul, sisi, koin atau task.
2. **Himpunan Solusi (S)** : kumpulan kandidat yang telah dipilih sebagai bagian dari solusi.
3. **Fungsi Seleksi** : memilih kandidat terbaik berdasarkan strategi greedy (heuristik).
4. **Fungsi Kelayakan** : memeriksa apakah kandidat layak dimasukkan ke dalam solusi.
5. **Fungsi Objektif** : digunakan untuk memaksimalkan atau meminimalkan hasil sesuai tujuan masalah.
6. **Fungsi Solusi** : menentukan apakah solusi akhir telah tercapai.

Proses pencarian dilakukan dengan memilih elemen dari Himpunan Kandidat (C) untuk dimasukkan ke dalam Himpunan Solusi (S), di mana (S) harus memenuhi kriteria kelayakan dan mengoptimalkan fungsi objektif. Rangkaian keputusan ini menghasilkan solusi optimum lokal, yang diharapkan bermuara pada solusi optimum global.

Namun, perlu dicatat bahwa solusi optimum global yang diperoleh dari Algoritma *Greedy* belum tentu merupakan solusi terbaik yang mutlak (bisa saja berupa solusi sub-optimum atau semu). Hal ini disebabkan oleh dua faktor utama:

1. Algoritma *Greedy* tidak mengevaluasi seluruh kemungkinan kombinasi solusi secara menyeluruh (*exhaustive search*).
2. Hasil akhir sangat bergantung pada pemilihan fungsi seleksi yang tepat.

Sebagai kesimpulan, **Algoritma Greedy** tidak selalu mampu memberikan solusi terbaik untuk semua jenis permasalahan. Namun, jika solusi mutlak tidak menjadi keharusan, algoritma ini sangat efektif digunakan untuk menghasilkan solusi hampiran (*approximation*). Pendekatan ini jauh lebih efisien dibandingkan algoritma lain seperti *brute force* yang membutuhkan waktu komputasi eksponensial (sangat lama) untuk masalah kompleks seperti *Traveling Salesman Problem* (TSP).

Dalam penerapannya, terdapat cukup banyak jenis permasalahan yang dapat diselesaikan menggunakan pendekatan greedy, diantaranya adalah sebagai berikut :

1. Persoalan penukaran uang (*coin exchange problem*)
2. Persoalan memilih aktivitas (*activity selection problem*)
3. Minimasi waktu di dalam sistem
4. Persoalan knapsack (*knapsack problem*)
5. Penjadwalan job dengan tenggat waktu (*job scheduling with deadlines*)
6. Pohon merentang minimum (*minimum spanning tree*)
7. Lintasan terpendek (*shortest path*)
8. Kode Huffman (*huffman code*)
9. Pecahan Mesir (*egyptian fraction*)

2.2 Cara Kerja Program Secara Umum

Program bot bekerja dengan sistem loop yang terus berjalan selama pertandingan berlangsung. Pada setiap iterasi, bot akan membaca kondisi arena, menentukan target, menghitung gerakan terbaik menggunakan algoritma greedy, lalu melakukan aksi seperti bergerak, mengunci radar, dan menembak.

Secara umum, cara kerja bot dapat dijelaskan sebagai berikut:

1. Bot melakukan scanning untuk mencari musuh
2. Data musuh disimpan, seperti posisi, energi, arah, dan kecepatan
3. Bot menghitung skor dari setiap kemungkinan aksi
4. Bot memilih aksi dengan nilai terbaik menggunakan pendekatan greedy
5. Bot bergerak menuju posisi terbaik
6. Bot mengarahkan radar dan senjata ke musuh
7. Bot melakukan tembakan jika kondisi memungkinkan
8. Proses diulang terus sampai pertandingan selesai

2.3 Implementasi Algoritma Greedy pada Bot

Implementasi algoritma greedy dilakukan dengan memilih tindakan yang memiliki nilai terbaik berdasarkan kondisi arena saat itu.

2.3.1 Pemilihan Target Musuh

Bot melakukan evaluasi terhadap setiap musuh yang terdeteksi menggunakan fungsi penilaian tertentu.

Penilaian dilakukan berdasarkan:

- a. Jarak musuh
- b. Energi musuh
- c. Konsistensi target sebelumnya

Musuh yang memiliki nilai tertinggi akan dipilih sebagai target utama.

Contoh strategi greedy yang digunakan:

1. Musuh dengan jarak lebih dekat akan diprioritaskan
2. Musuh dengan energi lebih kecil akan lebih diutamakan
3. Target lama tetap diprioritaskan agar serangan lebih konsisten

Dengan pendekatan tersebut, bot dapat memilih target yang paling menguntungkan untuk diserang.

2.3.2 Penentuan Arah Gerak

Bot menggunakan algoritma greedy untuk menentukan arah gerak terbaik. Program membuat beberapa kandidat arah gerakan, kemudian setiap arah akan diberikan skor.

Kriteria penilaian arah gerak meliputi:

1. Jarak terhadap dinding arena
2. Posisi relatif terhadap musuh
3. Jarak ideal dengan musuh
4. Kondisi energi bot
5. Kondisi energi musuh
6. Posisi aman untuk menghindari peluru

Bot kemudian memilih arah dengan skor tertinggi sebagai langkah terbaik

Sebagai contoh:

1. Jika bot terlalu dekat dengan dinding, maka arah yang mendekati dinding akan diberi penalti
2. Jika energi bot rendah, maka bot akan memilih posisi yang lebih defensif
3. Jika energi musuh rendah, bot akan lebih agresif mendekati target

Pendekatan ini memungkinkan bot bergerak secara adaptif sesuai kondisi pertandingan

2.3.3 Sistem Penghindaran Serangan

Bot memiliki mekanisme perubahan arah secara dinamis untuk menghindari peluru musuh. Ketika bot terkena peluru atau mendeteksi kemungkinan serangan, arah pergerakan akan diubah.

Strategi greedy diterapkan dengan memilih arah yang paling aman berdasarkan:

- a. Sudut terhadap musuh
- b. Jarak dari posisi bahaya
- c. Keamanan posisi terhadap dinding

Dengan cara ini, kemungkinan terkena peluru dapat dikurangi.

2.3.4 Sistem Penembakan

Bot menggunakan perhitungan greedy untuk menentukan:

- a. Kekuatan peluru
- b. Waktu menembak
- c. Prediksi posisi musuh

Kekuatan peluru ditentukan berdasarkan jarak dan kondisi energi. Jika musuh dekat, maka bot menggunakan peluru yang lebih kuat. Sebaliknya, jika musuh jauh atau bergerak cepat, maka bot menggunakan peluru dengan kekuatan lebih kecil agar lebih akurat dan hemat energi.

Bot juga melakukan prediksi posisi musuh menggunakan arah dan kecepatan gerakan target. Prediksi ini membantu meningkatkan peluang peluru mengenai lawan.

2.3.5 Cara Menjalankan Bot

Langkah-langkah menjalankan bot adalah sebagai berikut:

- a. Jalankan file jar aplikasi *GUI*
- b. Setup konfigurasi booter
 1. Klik tombol “Config”
 2. Klik tombol “Bot Root Directories”
 3. Masukkan *directory* yang berisi folder-folder bot
- c. Jalankan sebuah *battle*
 1. Klik tombol “Battle”
 2. Klik tombol “Start Battle”
 3. Setelah itu, muncul konfigurasi permainan
 - a. Bot-bot di dalam *directory* yang telah di setup pada proses konfigurasi akan otomatis muncul pada kotak di kiri atas
- d. Boot bot yang ingin dimainkan
 1. Select bot yang ingin dimainkan pada kotak di kiri atas
 2. Klik tombol “Boot →”
 3. Bot yang telah di select akan muncul pada kotak kanan atas
 4. Bot yang berhasil di-boot akan muncul pada kotak di kiri bawah
- e. Tambahkan bot ke dalam permainan
 1. Select bot yang ingin ditambahkan ke dalam permainan pada kotak di kiri bawah
 2. Klik tombol “Add →”
 3. Jika ingin menambahkan semua bot, klik tombol “Add all →”
 4. Bot yang telah ditambahkan akan otomatis muncul pada kotak di kanan bawah
- f. Mulai permainan dengan menekan tombol “Start Battle”
- g. Saat pertandingan berlangsung, bot akan bergerak dan mengambil keputusan secara otomatis menggunakan algoritma greedy.

BAB III

APLIKASI STRATEGI *GREEDY*

3.1 Mapping Persoalan Robocode Tank Royale ke Elemen-Elemen Algoritma *Greedy*

Tabel 1. Mapping persoalan Robocode

Elemen <i>Greedy</i>	Implementasi pada Bot Robocode Tank Royale
Himpunan Kandidat (C)	Semua kemungkinan tindakan yang dapat dipilih bot selama permainan berlangsung, seperti berpindah posisi, memutar radar, menyerang lawan, menghindari peluru, atau mengganti target serangan.
Himpunan Solusi (S)	Sekumpulan aksi yang akhirnya diputuskan dan dijalankan bot berdasarkan hasil evaluasi kondisi arena pada setiap waktu tertentu.
Fungsi Solusi	Kondisi akhir ketika strategi greedy menghasilkan hasil pertandingan, baik berupa kemenangan, kekalahan, maupun berakhirnya ronde permainan.
Fungsi Seleksi	Digunakan untuk menentukan pilihan aksi yang paling menguntungkan, misalnya memilih lawan dengan peluang serangan terbaik atau menentukan jalur gerak yang paling aman.
Fungsi Kelayakan	Berfungsi mengecek apakah aksi yang dipilih dapat dilakukan, seperti memastikan bot tidak keluar arena, tidak menabrak dinding, dan masih memiliki energi yang cukup untuk menyerang.
Fungsi Objektif	Bertujuan meningkatkan peluang kemenangan dengan menjaga ketahanan bot, mengurangi energi musuh, serta memaksimalkan efektivitas serangan dan pergerakan.

3.2 Eksplorasi Alternatif Solusi *Greedy* dalam persoalan Robocode Tank Royale

3.2.1 Alternatif 1 - TANK Q

Strategi greedy pada bot TANK_Q menggunakan kombinasi pemilihan target dan pergerakan adaptif. Bot memilih musuh berdasarkan skor tertinggi yang dihitung dari jarak dan energi musuh. Selain itu, bot mengevaluasi beberapa kandidat arah gerak untuk memilih posisi paling aman.

Heuristic

1. Memprioritaskan musuh terdekat.
2. Memprioritaskan musuh dengan energi rendah.
3. Memilih sudut gerak paling aman.
4. Menghindari dinding arena.

3.2.2 Alternatif 2 - BotKill

Strategi greedy pada BotKill berfokus pada pemilihan musuh dengan jarak paling dekat. Bot menggunakan pergerakan menyamping dan langkah pendek untuk menghindari serangan lawan.

Heuristic

1. Memilih target terdekat.
2. Menggunakan firepower berdasarkan jarak.
3. Bergerak menjauh dari pusat ancaman.
4. Menggunakan gerakan acak sederhana.

3.2.3 Alternatif 3 - BotViper

Strategi greedy pada BotViper berfokus pada duel 1 vs 1 menggunakan predictive aiming dan reactive dodging. Bot mendeteksi kapan musuh menembak berdasarkan penurunan energi lawan.

Heuristic

1. Menggunakan prediksi arah gerak musuh.
2. Bergerak tegak lurus terhadap lawan.
3. Menghindar saat musuh menembak.
4. Menggunakan micro-movement acak.

3.2.4 Alternatif 4 - Velora

Strategi greedy pada VELORA berfokus pada opportunistic attack dan survival movement. Bot menyerang agresif ketika peluang tinggi, namun akan lebih defensif saat energi rendah.

Heuristic

1. Menggunakan firepower berdasarkan kondisi musuh.
2. Bergerak secara acak agar sulit diprediksi.
3. Mengutamakan survival saat energi rendah.
4. Menghindari tabrakan dengan dinding dan musuh.

3.3 Analisis Efisiensi dan Efektivitas Alternatif Solusi Greedy

Keempat alternatif strategi greedy memiliki karakteristik yang berbeda dalam pengambilan keputusan, pergerakan, maupun sistem penyerangan. Perbedaan heuristic pada setiap bot mempengaruhi tingkat efisiensi dan efektivitas selama pertandingan berlangsung.

Tabel 2. Analisis efisiensi dan efektivitas

Alternatif	Efisiensi	Efektivitas	Analisis
TANK-Q	Sedang	Tinggi	Menggunakan evaluasi target dan kandidat gerakan sehingga keputusan lebih optimal, namun membutuhkan perhitungan lebih kompleks.
BotKill	Tinggi	Sedang	Strategi sederhana dan cepat karena hanya memprioritaskan musuh terdekat, tetapi kurang efektif terhadap musuh dengan pergerakan kompleks.
BotViper	Sedang	Tinggi	Memiliki predictive aiming dan reactive dodging yang efektif untuk duel 1 vs 1, namun membutuhkan tracking musuh yang stabil.
Velora	Tinggi	Sedang	Menggunakan gerakan acak dan opportunistic attack sehingga cukup ringan dijalankan, tetapi akurasi serangan lebih rendah karena tidak menggunakan prediksi target.

Berdasarkan hasil analisis, strategi **TANK_Q** dan **BotViper** memiliki efektivitas yang lebih tinggi karena menggunakan evaluasi kondisi arena dan prediksi gerakan lawan. Namun, kedua strategi tersebut membutuhkan proses perhitungan yang lebih kompleks dibanding strategi lainnya.

Sementara itu, **BotKill** dan **VELORA** memiliki efisiensi yang lebih tinggi karena menggunakan heuristic yang lebih sederhana sehingga respon bot lebih cepat dan penggunaan resource lebih ringan. Akan tetapi, efektivitasnya cenderung lebih rendah ketika menghadapi lawan dengan strategi yang lebih adaptif.

Secara keseluruhan, setiap alternatif memiliki kelebihan dan kekurangan masing-masing tergantung kondisi pertandingan dan heuristic greedy yang digunakan.

3.4 Strategi Greedy yang Dipilih

Dari empat alternatif solusi greedy yang telah dianalisis, strategi yang dipilih untuk diimplementasikan dalam program adalah strategi pada bot **TANK_Q**.

Strategi ini dipilih karena memiliki keseimbangan terbaik antara kemampuan menyerang, bertahan, dan pengambilan keputusan secara adaptif selama pertandingan berlangsung.

Alasan Pemilihan Strategi

1. **Pemilihan Target Lebih Optimal.** **TANK_Q** tidak hanya memilih musuh terdekat, tetapi juga mempertimbangkan energi musuh dan konsistensi target. Hal ini membuat serangan menjadi lebih efektif.

2. **Pergerakan Lebih Adaptif**, Bot mengevaluasi beberapa kandidat arah gerakan sebelum menentukan posisi terbaik. Pendekatan ini membantu bot menghindari dinding dan mengurangi kemungkinan terkena peluru.
3. **Menggunakan Predictive Aiming**, Sistem penembakan menggunakan prediksi posisi musuh sehingga peluang tembakan mengenai target menjadi lebih tinggi dibanding strategi tembakan langsung.
4. **Memiliki Sistem Evasive Movement**, Bot dapat mengganti arah gerakan secara dinamis ketika terkena peluru atau mendeteksi lawan menembak, sehingga kemampuan bertahan menjadi lebih baik.
5. **Keseimbangan Agresif dan Defensif**, Strategi TANK_Q mampu menyesuaikan perilaku berdasarkan kondisi pertandingan, sehingga tidak terlalu pasif maupun terlalu agresif.

Pertimbangan Pemilihan

Meskipun strategi TANK_Q memiliki kompleksitas lebih tinggi dibanding alternatif lainnya, efektivitas yang dihasilkan juga lebih baik. Bot mampu mengambil keputusan secara lebih cerdas karena mempertimbangkan banyak faktor dalam proses greedy, seperti jarak, energi musuh, posisi arena, dan keamanan pergerakan.

Oleh karena itu, strategi greedy pada TANK_Q dipilih sebagai strategi utama yang diimplementasikan dalam program Robocode Tank Royale.

BAB IV

IMPLEMENTASI DAN PENGUJIAN

4.1 Implementasi Keempat Alternatif Solusi

4.1.1 Pseudocode Alternatif 1: TANK Q

Mulai

Inisialisasi data target:

```
targetId ← -1
targetX ← tidak valid
targetY ← 0
targetEnergy ← 100
targetDirection ← 0
targetSpeed ← 0
targetScore ← nilai sangat kecil
lastSeenTurn ← nilai sangat lama
```

Inisialisasi data gerakan:

```
moveDirection ← 1
panicUntil ← 0
lastDirectionChange ← 0
```

Atur warna **body**, turret, radar, **bullet**, scan, tracks, dan gun

Atur gun dan radar agar dapat **bergerak** independen dari **body**

WHILE bot masih berjalan DO

IF arah gerak sudah terlalu lama tidak **berubah** THEN

```
moveDirection ← moveDirection × -1
lastDirectionChange ← turn saat ini
```

ENDIF

IF target masih valid THEN

```
Hitung sudut radar terhadap target
Putar radar ke arah target
```

Hitung enemyAngle berdasarkan posisi target

Bentuk daftar kandidat arah gerak:

enemyAngle + 90
enemyAngle - 90
enemyAngle + 120
enemyAngle - 120
enemyAngle + 45
enemyAngle - 45
arah menuju tengah arena

bestScore ← nilai sangat kecil

bestAngle ← kandidat arah pertama

FOR setiap angle dalam daftar kandidat arah DO

Prediksi posisi bot jika bergerak ke angle:

predictedX ← posisi X bot + $\cos(\text{angle}) \times \text{jarak prediksi}$

predictedY ← posisi Y bot + $\sin(\text{angle}) \times \text{jarak prediksi}$

score ← 0

IF predictedX atau predictedY terlalu dekat dengan dinding THEN

score ← score - 700

ELSE

score ← score + 180

ENDIF

Hitung predictedDistance dari posisi prediksi bot ke target

idealDistance ← 330

score ← score - $|\text{predictedDistance} - \text{idealDistance}|$

Hitung sideAngle sebagai sudut gerak relatif terhadap musuh

IF bot sedang dalam mode panik THEN

IF sideAngle berada di antara 70 sampai 130 derajat THEN

score ← score + 310

ENDIF

ENDIF

Hitung preferredOrbit berdasarkan arah musuh dan moveDirection

Hitung orbitError dari angle terhadap preferredOrbit

```
score ← score + 80 - orbitError × 0.4
```

Tambahkan nilai acak kecil ke score agar gerakan tidak mudah ditebak

```
IF score > bestScore THEN  
    bestScore ← score  
    bestAngle ← angle  
ENDIF
```

```
ENDFOR
```

```
IF bot sedang dalam mode panik THEN  
    moveAmount ← 185  
ELSE  
    moveAmount ← 145  
ENDIF
```

Hitung sudut belok menuju bestAngle

```
IF sudut belok lebih besar dari 90 derajat THEN  
    Kurangi sudut belok sebesar 180 derajat  
    moveAmount ← moveAmount × -1  
ELSE IF sudut belok lebih kecil dari -90 derajat THEN  
    Tambahkan sudut belok sebesar 180 derajat  
    moveAmount ← moveAmount × -1  
ENDIF
```

Putar body sesuai sudut belok
Gerakkan bot sejauh moveAmount

Hitung distance dari bot ke target

```
IF energi bot < 10 THEN  
    firePower ← 0.6  
ELSE IF energi target < 8 DAN distance < 450 THEN  
    firePower ← 1.8  
ELSE IF distance < 150 THEN  
    firePower ← 3.0  
ELSE IF distance < 300 THEN  
    firePower ← 2.1  
ELSE IF distance < 500 THEN
```

```

        firePower ← 1.3
    ELSE
        firePower ← 0.7
    ENDIF

    IF energi bot cukup untuk menembak THEN

        Hitung kecepatan peluru berdasarkan firePower
        Hitung waktu tempuh peluru menuju target

        predictedX ← targetX
        predictedY ← targetY

        FOR dua kali iterasi DO
            predictedX ← targetX + cos(targetDirection) × targetSpeed
            predictedY ← targetY + sin(targetDirection) × targetSpeed

            Batasi predictedX agar tetap berada di dalam arena
            Batasi predictedY agar tetap berada di dalam arena

            Perbarui waktu tempuh berdasarkan jarak bot ke posisi
prediksi
        ENDFOR

        Hitung sudut gun menuju posisi prediksi target
        Putar gun ke arah posisi prediksi target

        IF distance < 180 THEN
            aimTolerance ← 8
        ELSE IF distance < 350 THEN
            aimTolerance ← 5
        ELSE
            aimTolerance ← 3
        ENDIF

        IF sudut gun lebih kecil dari aimTolerance DAN gun heat = 0
THEN
            Tembak target dengan firePower
        ENDIF
    
```

```

ENDIF

ELSE

    Hapus target lama
    Reset skor target
    Putar radar 360 derajat
    Putar body sesuai moveDirection
    Gerakkan bot maju untuk mencari musuh

ENDIF

Jalankan seluruh perintah pada turn tersebut

ENDWHILE

EVENT ketika radar mendeteksi musuh:

    Hitung distance dari bot ke musuh

    closeScore ← 700 - distance
    weakEnemyScore ← 100 - energi musuh

    IF musuh yang terdeteksi adalah target yang sama THEN
        sameTargetBonus ← 90
    ELSE
        sameTargetBonus ← 0
    ENDIF

    score ← closeScore + weakEnemyScore × 3 + sameTargetBonus

    IF belum ada target ATAU musuh adalah target yang sama ATAU score > targetScore THEN

        IF musuh adalah target yang sama THEN
            energyDrop ← energi target sebelumnya - energi musuh saat ini

            IF energyDrop > 0 DAN energyDrop ≤ 3 THEN
                moveDirection ← moveDirection × -1
                panicUntil ← turn saat ini + 18
                lastDirectionChange ← turn saat ini
            ENDIF
        ENDIF
    ENDIF

```

```

ENDIF
ENDIF

Simpan musuh sebagai target:
    targetId ← id musuh
    targetX ← posisi X musuh
    targetY ← posisi Y musuh
    targetEnergy ← energi musuh
    targetDirection ← arah musuh
    targetSpeed ← kecepatan musuh
    targetScore ← score
    lastSeenTurn ← turn saat ini

ENDIF

EVENT ketika bot terkena peluru:

    moveDirection ← moveDirection × -1
    panicUntil ← turn saat ini + 22
    lastDirectionChange ← turn saat ini

EVENT ketika bot menabrak dinding:

    moveDirection ← moveDirection × -1
    panicUntil ← turn saat ini + 20
    lastDirectionChange ← turn saat ini

EVENT ketika bot menabrak bot lain:

    moveDirection ← moveDirection × -1
    panicUntil ← turn saat ini + 16

    Hitung sudut gun menuju bot yang ditabrak
    Putar gun ke arah bot tersebut

    IF sudut gun cukup kecil DAN gun heat = 0 DAN energi bot > 6 THEN
        Tembak dengan firePower 2.5
    ENDIF

```

Selesai

4.1.2 Pseudocode Alternatif 2: BotKill

Mulai

Inisialisasi:

```
targetId ← tidak ada  
minDistance ← nilai sangat besar  
random ← generator angka acak
```

Atur warna body, turret, dan radar

WHILE bot masih berjalan DO

```
Reset targetId menjadi tidak ada  
Reset minDistance menjadi nilai sangat besar  
  
Putar radar 360 derajat untuk mencari seluruh musuh di sekitar bot
```

ENDWHILE

EVENT ketika radar mendeteksi musuh:

Hitung selisih posisi musuh terhadap bot:

```
dx ← posisiX musuh - posisiX bot  
dy ← posisiY musuh - posisiY bot
```

Hitung distance:

```
distance ← akar( $dx^2 + dy^2$ )
```

IF belum ada target OR distance < minDistance THEN

```
targetId ← id musuh yang terdeteksi  
minDistance ← distance
```

Hitung directionToEnemy berdasarkan posisi musuh

```

Hitung sudut putar gun menuju musuh:
    gunTurn ← selisih arah gun dengan directionToEnemy

Putar gun ke arah musuh

IF distance < 150 THEN
    Tembak dengan firePower 3
ELSE IF distance < 300 THEN
    Tembak dengan firePower 2
ELSE IF distance < 500 THEN
    Tembak dengan firePower 1
ELSE
    Tidak menembak untuk menghemat energi
ENDIF

Hitung bearingToEnemy:
    bearingToEnemy ← selisih arah body dengan directionToEnemy

Hitung turnAngle:
    turnAngle ← bearingToEnemy + 105

Putar body sebesar turnAngle
// Sudut 105 derajat digunakan agar bot bergerak melebar,
// tidak maju lurus ke arah musuh.

IF nilai random menghasilkan 0 THEN
    Maju sejauh 80
ELSE
    Mundur sejauh 80
ENDIF

ENDIF

EVENT ketika bot menabrak bot lain:

    Mundur sejauh 120
    Putar kanan 90 derajat
    // Bot menjauh dari kerumunan agar tidak menjadi target mudah.

EVENT ketika bot menabrak dinding:

```

```
Mundur sejauh 150
Putar kanan 120 derajat
// Bot keluar dari area dinding agar tidak terjebak.
```

Selesai

4.1.3 Pseudocode Alternatif 3: BotViper

Mulai

Inisialisasi:

```
previousEnemyEnergy ← 100
moveDirection ← 1
random ← generator angka acak
```

Atur warna body, turret, dan radar

WHILE bot masih berjalan DO

```
Putar radar 360 derajat secara terus-menerus
// Radar terus diputar agar musuh tetap terdeteksi dalam duel 1 vs 1
```

ENDWHILE

EVENT ketika radar mendeteksi musuh:

Hitung directionToEnemy berdasarkan posisi musuh

Hitung jarak musuh:

```
dx ← posisiX musuh - posisiX bot
dy ← posisiY musuh - posisiY bot
distance ← akar(dx2 + dy2)
```

Prediksi arah tembakan:

```
predictedDirection ← directionToEnemy + (kecepatan musuh × 0.6)
```

Hitung sudut putar gun menuju predictedDirection
Putar gun ke arah prediksi musuh

```
IF distance < 200 THEN
    Tembak dengan firePower 3
ELSE IF distance < 450 THEN
    Tembak dengan firePower 2
ELSE
    Tembak dengan firePower 1
ENDIF
```

Hitung penurunan energi musuh:
energyDrop $\leftarrow$ previousEnemyEnergy - energi musuh saat ini

Perbarui previousEnemyEnergy:
previousEnemyEnergy $\leftarrow$ energi musuh saat ini

Hitung bearingToEnemy:
bearingToEnemy $\leftarrow$ selisih arah body dengan directionToEnemy

Putar body agar tegak lurus terhadap musuh:
turnAngle $\leftarrow$ bearingToEnemy + 90

```
IF energyDrop > 0 DAN energyDrop  $\leq$  3 THEN
    // Musuh dianggap baru menembak
```

Balik arah gerak:
moveDirection $\leftarrow$ moveDirection $\times$ -1

Tentukan jarak menghindari secara acak:
evadeDistance $\leftarrow$ 50 + nilai acak antara 0 sampai 39

```
IF moveDirection > 0 THEN
    Maju sejauh evadeDistance
ELSE
    Mundur sejauh evadeDistance
ENDIF
```

```
ELSE
    // Jika musuh tidak terdeteksi menembak,
    // bot tetap melakukan micro-movement kecil
```

```

    IF nilai acak lebih besar dari 7 THEN
        IF moveDirection > 0 THEN
            Maju sejauh 30
        ELSE
            Mundur sejauh 30
        ENDIF
    ENDIF

ENDIF

EVENT ketika bot menabrak bot lain:

    Balik arah gerak
    Mundur untuk menjaga jarak ideal duel

EVENT ketika bot menabrak dinding:

    Balik arah gerak
    Bergerak menjauh dari dinding
    Putar kanan 90 derajat agar tidak terjebak di sudut arena

Selesai

```

4.1.4 Pseudocode Alternatif 4: Velora

```

Mulai

Inisialisasi:
    random ← generator angka acak
    moveDirection ← 1

Atur warna body, turret, radar, bullet, dan scan

Putar radar secara terus-menerus untuk mencari musuh

WHILE bot masih berjalan DO

```

```

IF bot sedang tidak bergerak THEN
    Gerakkan bot maju sejauh 150 × moveDirection
    Putar body secara acak antara -60 sampai 60 derajat
ENDIF

IF energi bot < 30 THEN
    Putar body sebesar 45 derajat
    Gerakkan bot maju sejauh 200 × moveDirection
    // Saat energi rendah, prioritas berubah menjadi bertahan
ENDIF

```

Jalankan seluruh perintah pada turn tersebut

ENDWHILE

EVENT ketika radar mendeteksi musuh:

```

Hitung enemyDirection berdasarkan posisi musuh
Hitung distance dari bot ke musuh

```

```

Arahkan gun langsung ke posisi musuh saat ini
Arahkan radar langsung ke posisi musuh saat ini

```

```

firePower ← 1.0

```

```

IF energi musuh < 15 OR distance < 200 THEN
    firePower ← 3.0
    // Gunakan tembakan maksimal jika musuh lemah atau sangat dekat
ELSE IF distance < 500 THEN
    firePower ← 2.0
    // Gunakan tembakan sedang untuk jarak menengah
ELSE
    firePower ← 1.0
    // Gunakan tembakan kecil untuk jarak jauh agar hemat energi
ENDIF

```

```

IF gun heat = 0 THEN
    Tembak dengan firePower
ENDIF

```

```

IF distance < 150 THEN
    moveDirection ← moveDirection × -1
    Gerakkan bot menjauh sejauh 150 × moveDirection
    // Mundur untuk menghindari tabrakan dan ram damage
ENDIF

```

EVENT ketika bot menabrak dinding:

```

moveDirection ← moveDirection × -1
Gerakkan bot sejauh 100 × moveDirection
Putar body sebesar 90 derajat
// Bot memantul keluar dari batas arena

```

EVENT ketika bot terkena peluru:

```

moveDirection ← moveDirection × -1
Putar body secara acak antara -90 sampai 90 derajat
Gerakkan bot sejauh 150 × moveDirection
// Reaksi acak agar tidak tertembak di posisi yang sama

```

EVENT ketika bot menabrak bot lain:

```

moveDirection ← moveDirection × -1
Gerakkan bot sejauh 100 × moveDirection
// Bot menjauh agar tidak terus berada dalam kontak fisik

```

Selesai

4.2 Penjelasan Fungsi, Struktur Data, dan Prosedur

Bagian ini menjelaskan struktur data, fungsi, dan prosedur yang digunakan pada bot TANK Q sebagai bot utama. Bot ini menerapkan strategi Greedy Target Scoring & Multi-Candidate Greedy Movement, yaitu strategi greedy yang memilih keputusan terbaik berdasarkan kondisi pertandingan pada setiap turn. Keputusan utama yang diambil bot meliputi pemilihan target, pemilihan arah gerak, penghindaran serangan, dan penentuan kekuatan tembakan.

4.2.1 Struktur Data

Implementasi TANK Q menggunakan struktur data berupa class TANK_Q yang mewarisi class Bot dari library Robocode Tank Royale. Class ini menjadi pusat pengaturan seluruh perilaku bot selama pertandingan, mulai dari scanning, pemilihan target, pergerakan, hingga penembakan. Struktur data utama yang digunakan dapat dibagi menjadi dua bagian, yaitu data target dan data gerakan.

1. Data Target

```
private int targetId = -1;
private double targetX = double.NaN;
private double targetY;
private double targetEnergy = 100;
private double targetDirection;
private double targetSpeed;
private double targetScore = double.NegativeInfinity;
private int lastSeenTurn = -9999;
```

Variabel targetId digunakan untuk menyimpan identitas musuh yang sedang dijadikan target. Nilai awalnya adalah -1, yang menunjukkan bahwa bot belum memiliki target.

Variabel targetX dan targetY digunakan untuk menyimpan posisi koordinat musuh. Data ini dipakai untuk mengarahkan radar, mengarahkan senjata, menghitung jarak, dan menentukan arah gerakan.

Variabel targetEnergy menyimpan energi musuh. Data ini dipakai untuk menentukan prioritas target dan mendeteksi kemungkinan musuh baru saja menembak. Jika energi musuh turun dalam rentang tertentu, bot menganggap musuh baru menembakkan peluru.

Variabel targetDirection dan targetSpeed digunakan untuk menyimpan arah dan kecepatan musuh. Keduanya digunakan pada proses prediksi posisi musuh saat bot akan menembak.

Variabel targetScore menyimpan nilai evaluasi target. Skor ini dihitung berdasarkan jarak musuh, energi musuh, dan bonus jika musuh tersebut adalah target yang sama. Semakin tinggi skor target, semakin besar peluang musuh tersebut dipilih sebagai target utama.

Variabel lastSeenTurn menyimpan turn terakhir ketika target terlihat oleh radar. Variabel ini digunakan untuk mengecek apakah target masih valid atau sudah terlalu lama hilang dari pemindaian.

2. Data Gerakan

```
private double moveDirection = 1;
private int panicUntil = 0;
private int lastDirectionChange = 0;
```

Variabel `moveDirection` digunakan untuk menentukan arah gerak bot. Nilainya dapat berubah menjadi 1 atau -1. Ketika bot terkena peluru, mendeteksi musuh menembak, atau menabrak dinding, nilai ini dibalik agar bot tidak bergerak dengan pola yang mudah ditebak.

Variabel `panicUntil` digunakan untuk menentukan batas waktu mode panik. Mode panik aktif ketika bot terkena serangan atau mendeteksi musuh baru menembak. Saat mode ini aktif, bot lebih memprioritaskan gerakan menyamping untuk menghindari peluru.

Variabel `lastDirectionChange` menyimpan turn terakhir saat bot mengganti arah gerak. Variabel ini membantu bot mengubah arah secara berkala agar pola pergerakannya tidak monoton.

4.2.2 Fungsi Utama

1. Fungsi `Main()`

```
static void Main(string[] args)
{
    new TANK_Q().Start();
}
```

Fungsi ini merupakan titik awal eksekusi program. Fungsi `Main()` membuat objek `TANK_Q`, kemudian menjalankan bot di arena Robocode Tank Royale.

2. Konstruktor `TANK_Q()`

```
TANK_Q() : base(BotInfo.FromFile("TANK_Q.json")) { }
```

Konstruktor digunakan untuk memuat informasi konfigurasi bot dari file `TANK_Q.json`. File ini berisi data identitas bot yang dibutuhkan saat bot dijalankan.

3. Fungsi `Run()`

```
public override void Run()
```

Fungsi `Run()` adalah fungsi utama yang terus berjalan selama pertandingan berlangsung. Di dalam fungsi ini, bot akan mengecek apakah target masih valid. Jika target masih valid, bot akan mengunci radar, memilih gerakan terbaik, dan melakukan penembakan. Jika target tidak valid, bot masuk ke mode pencarian musuh. Pada fungsi ini juga dipanggil `GreedyDirectionShuffle()` agar arah gerak bot berubah secara berkala dan tidak mudah diprediksi. Pada fungsi ini juga dipanggil `GreedyDirectionShuffle()` agar arah gerak bot berubah secara berkala dan tidak mudah diprediksi.

4. Fungsi OnScannedBot()

```
public override void OnScannedBot(ScannedBotEvent e)
```

Fungsi ini dipanggil ketika radar mendeteksi musuh. Pada fungsi ini, bot menghitung skor musuh menggunakan GreedyTargetScore(). Jika musuh yang terdeteksi memiliki skor lebih tinggi atau merupakan target yang sama, maka data target akan diperbaharui.

Fungsi ini juga digunakan untuk mendeteksi kemungkinan musuh menembak. Jika energi musuh turun antara 0 sampai 3 poin, bot menganggap musuh baru saja menembak. Sebagai respons, bot membalik arah gerak dan mengaktifkan mode panik.

5. Fungsi GreedyTargetScore()

```
private double GreedyTargetScore(ScannedBotEvent e)
```

Fungsi ini digunakan untuk menghitung skor target berdasarkan jarak musuh, energi musuh, dan bonus apabila musuh tersebut adalah target yang sama. Musuh yang lebih dekat akan mendapat skor lebih tinggi. Musuh dengan energi rendah juga lebih diprioritaskan karena lebih mudah dikalahkan.

6. Fungsi HasFreshTarget()

```
private bool HasFreshTarget()
```

Fungsi ini digunakan untuk mengecek apakah target masih valid. Target dianggap valid jika data posisi target masih tersedia dan target tidak hilang dari radar lebih dari batas turn tertentu.

7. Fungsi SearchMode()

```
private void SearchMode()
```

Fungsi ini dijalankan ketika bot belum memiliki target atau target sudah tidak valid. Bot akan menghapus target lama, memutar radar 360 derajat, memutar body, dan bergerak maju untuk mencari musuh.

8. Fungsi GreedyRadarLock()

```
private void GreedyRadarLock()
```

Fungsi ini digunakan untuk mengunci radar ke arah target. Dengan radar lock, bot dapat terus memperbarui informasi posisi, energi, arah, dan kecepatan musuh.

9. Fungsi GreedyMovement()

```
private void GreedyMovement()
```

Fungsi ini digunakan untuk memilih arah gerak terbaik. Bot membuat beberapa kandidat arah, seperti gerak menyamping, gerak serong, dan gerak menuju tengah arena. Setiap kandidat arah kemudian dievaluasi menggunakan GreedyMoveScore().

10. Fungsi GreedyMoveScore()

```
private double GreedyMoveScore(double angle)
```

Fungsi ini digunakan untuk memberi skor pada setiap kandidat arah gerak. Penilaian dilakukan berdasarkan beberapa faktor, yaitu jarak terhadap dinding, jarak ideal terhadap musuh, gerakan menyamping saat mode panik, pola orbit, dan sedikit nilai acak agar gerakan tidak mudah ditebak.

11. Fungsi GreedyAimAndFire()

```
private void GreedyAimAndFire()
```

Fungsi ini digunakan untuk mengarahkan senjata dan melakukan tembakan. Bot menghitung jarak ke target, memilih kekuatan peluru menggunakan GreedyFirePower(), lalu memprediksi posisi target berdasarkan arah dan kecepatan musuh.

12. Fungsi GreedyFirePower()

```
private double GreedyFirePower(double distance)
```

Fungsi ini menentukan kekuatan peluru berdasarkan jarak musuh dan kondisi energi. Jika energi bot rendah, bot menggunakan peluru kecil. Jika musuh dekat, bot menggunakan peluru kuat. Jika musuh jauh, bot menggunakan peluru kecil agar lebih hemat energi.

13. Fungsi GreedyDirectionShuffle()

```
private void GreedyDirectionShuffle()
```

Fungsi ini digunakan untuk mengubah arah gerak secara berkala. Tujuannya agar bot tidak bergerak dengan pola orbit yang terlalu monoton dan mudah dibaca lawan.

14. Fungsi SmartMove()

```
private void SmartMove(double wantedAngle, double distance)
```

Fungsi ini digunakan untuk mengatur pergerakan bot ke arah terbaik yang sudah dipilih. Jika sudut belok terlalu besar, bot akan memilih bergerak mundur agar lebih efisien daripada memutar body terlalu jauh.

15. Fungsi Event Tabrakan dan Serangan

Fungsi OnHitByBullet() dijalankan ketika bot terkena peluru. Bot langsung membalik arah gerak dan mengaktifkan mode panik. Fungsi OnHitWall() dijalankan ketika bot menabrak dinding. Bot membalik arah agar tidak terus berada di dekat batas arena. Fungsi OnHitBot() dijalankan ketika bot menabrak bot lain. Bot membalik arah, mengarahkan gun ke bot yang ditabrak, lalu menembak jika kondisi memungkinkan.

4.2.3 Prosedur

Prosedur kerja TANK_Q dimulai dari fungsi Run(). Pada awal pertandingan, bot mengatur warna body, turret, radar, bullet, scan, tracks, dan gun. Bot juga mengatur agar gun dan radar dapat bergerak secara independen dari body. Hal ini penting karena bot harus tetap bisa memindai dan menembak meskipun body sedang bergerak.

Selama pertandingan berlangsung, bot akan menjalankan loop utama. Pada setiap turn, bot terlebih dahulu memeriksa apakah arah gerak perlu diubah melalui GreedyDirectionShuffle(). Setelah itu, bot mengecek apakah target masih valid. Jika target masih valid, bot mengunci radar ke arah target, menghitung arah gerak terbaik melalui GreedyMovement(), lalu mengarahkan senjata dan menembak menggunakan GreedyAimAndFire(). Jika target tidak valid, bot masuk ke SearchMode(). Dalam mode ini, radar diputar 360 derajat dan bot bergerak maju sambil mencari musuh baru. Ketika musuh terdeteksi, event OnScannedBot() akan dijalankan. Pada event tersebut, bot menghitung skor musuh. Jika musuh memiliki skor lebih tinggi dari target sebelumnya, bot menyimpan musuh tersebut sebagai target baru. Strategi greedy terlihat jelas dalam tiga tahap utama. Pertama, pada pemilihan target, bot memilih musuh dengan skor tertinggi berdasarkan jarak, energi, dan konsistensi target. Kedua, pada pemilihan arah gerak, bot mengevaluasi beberapa kandidat arah dan memilih arah dengan skor terbaik. Ketiga, pada penembakan, bot memilih kekuatan peluru berdasarkan jarak dan kondisi energi. Jika bot terkena peluru, menabrak dinding, atau menabrak bot lain, bot akan langsung mengubah arah gerak. Respons ini bertujuan agar bot tidak terus berada pada posisi yang berbahaya. Seluruh proses dilakukan berulang hingga pertandingan selesai. Strategi greedy pada TANK_Q bersifat adaptif karena setiap keputusan diambil berdasarkan kondisi saat itu. Bot tidak menghitung seluruh kemungkinan strategi jangka panjang, tetapi memilih aksi lokal terbaik pada setiap turn. Pendekatan ini membuat bot mampu bereaksi cepat terhadap perubahan posisi musuh, kondisi energi, dan situasi arena.

4.3 Pengujian

Pengujian dilakukan untuk mengetahui performa bot utama TANK_Q dalam pertandingan Robocode Tank Royale. Bot utama diuji menggunakan format pertandingan 1 vs 1 melawan bot asisten. Pengujian dilakukan sebanyak tiga kali dengan lawan yang berbeda agar dapat melihat kemampuan bot dalam beberapa kondisi pertandingan. Pengujian ini berfokus pada tiga aspek utama, yaitu kemampuan menyerang, kemampuan bertahan, dan kemampuan mengambil keputusan greedy pada setiap turn. Strategi greedy yang diuji adalah Greedy Target Scoring & Multi-Candidate Greedy Movement, yaitu strategi yang memilih target berdasarkan skor, memilih arah gerak dari beberapa kandidat, dan menentukan kekuatan tembakan berdasarkan jarak serta kondisi energi.

Skenario pengujian yang digunakan adalah sebagai berikut:

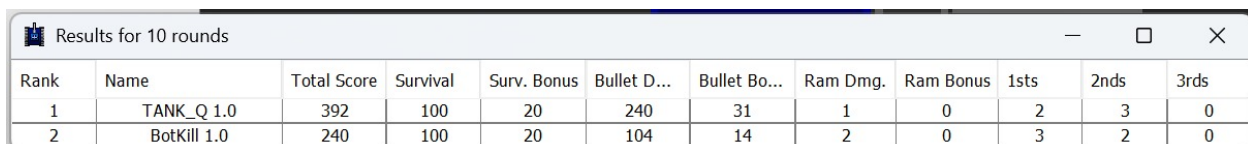
1. Bot utama yang diuji adalah **TANK_Q**.
2. Pengujian dilakukan dalam format **1 vs 1**.
3. Lawan yang digunakan adalah bot asisten.
4. Setiap pengujian dilakukan sebanyak **10 round**.

5. Data yang diamati meliputi skor akhir, hasil pertandingan, dan kejadian unik selama pertandingan.
6. Pengujian dilakukan untuk melihat kesesuaian strategi greedy dengan perilaku bot di arena

Tabel 3. Pengujian 1 vs 1 melawan Bot Asisten/Alternatif

No	Bot Lawan	Jumlah Round	Skor TANK_Q	Skor Bot Lawan	Hasil	Kejadian Unik
1	BotKill	10	392	240	Menang	TANK_Q cukup stabil menjaga jarak dan beberapa kali berhasil menembak pada jarak dekat.
2	BotViper	10	745	236	Menang	TANK_Q beberapa kali kehilangan target ketika lawan bergerak acak dan mendekati dinding.
3	VELORA	10	915	103	Menang	TANK_Q berhasil memanfaatkan perubahan arah ketika terkena peluru dan lebih sering bertahan sampai akhir ronde.

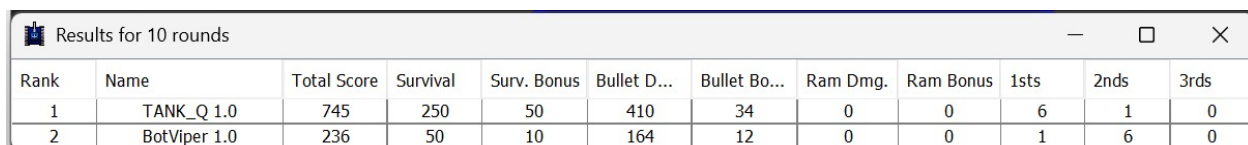
4.3.1 Hasil Pengujian: BotKill



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TANK_Q 1.0	392	100	20	240	31	1	0	2	3	0
2	BotKill 1.0	240	100	20	104	14	2	0	3	2	0

Gambar 2.1 Hasil pengujian melawan BotKill

4.3.2 Hasil Pengujian: BotViper



Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TANK_Q 1.0	745	250	50	410	34	0	0	6	1	0
2	BotViper 1.0	236	50	10	164	12	0	0	1	6	0

Gambar 2.2 Hasil pengujian melawan BotViper

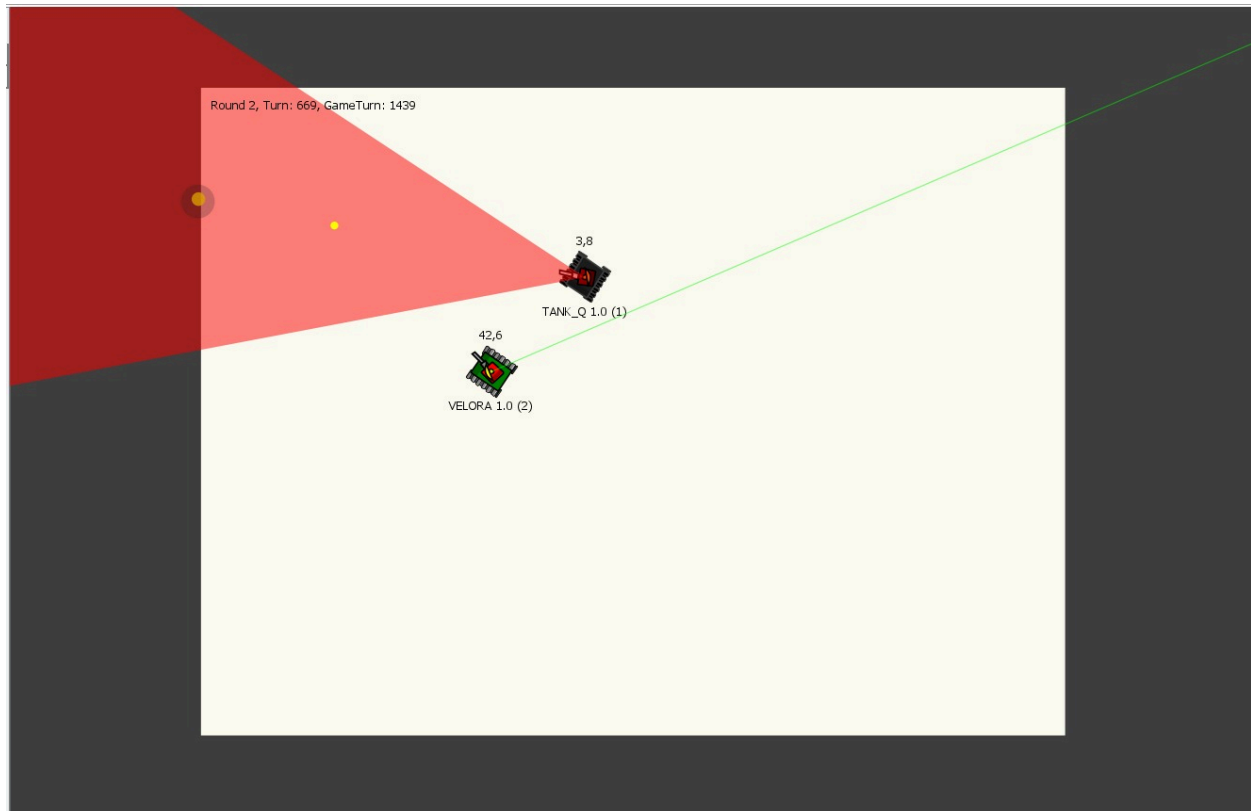
4.3.3 Hasil Pengujian: VELORA

Results for 10 rounds											
Rank	Name	Total Score	Survival	Surv. Bonus	Bullet D...	Bullet Bo...	Ram Dmg.	Ram Bonus	1sts	2nds	3rds
1	TANK_Q 1.0	915	300	60	491	62	1	0	7	1	0
2	VELORA 1.0	103	50	10	42	0	1	0	1	7	0

Gambar 2.3 Hasil pengujian melawan VELORA

4.3.4 Hasil Pengujian Mengalami Kejadian Unik

TANK_Q beberapa kali kehilangan target ketika lawan bergerak acak dan mendekati dinding.



Gambar 2.4 Kejadian unik

4.4 Analisis Hasil Pengujian

Berdasarkan tiga kali pengujian, strategi greedy pada TANK_Q menunjukkan hasil yang cukup baik. Dari tiga pertandingan, TANK_Q memperoleh dua kemenangan dan satu kekalahan. Hasil ini menunjukkan bahwa strategi greedy yang diterapkan mampu membantu bot mengambil keputusan secara cepat dan cukup efektif dalam kondisi tertentu.

Pada aspek **pemilihan target**, TANK_Q mampu menentukan target berdasarkan skor yang dihitung dari jarak, energi musuh, dan bonus target yang sama. Strategi ini membuat bot tidak menyerang secara acak.

Musuh yang lebih dekat dan memiliki energi lebih rendah akan lebih diprioritaskan karena dianggap lebih menguntungkan untuk diserang. Pemilihan target seperti ini sesuai dengan prinsip greedy, yaitu memilih pilihan terbaik pada kondisi saat itu.

Pada aspek **pergerakan**, TANK_Q menggunakan beberapa kandidat arah gerak. Setiap arah diberi skor berdasarkan keamanan dari dinding, jarak ideal terhadap musuh, dan kecocokan dengan pola orbit. Dalam pengujian pertama dan ketiga, strategi ini membantu bot menjaga posisi agar tidak terlalu sering menabrak dinding. Namun, pada pengujian kedua, bot masih sempat berada terlalu dekat dengan batas arena sehingga ruang geraknya menjadi terbatas.

Pada aspek **penembakan**, TANK_Q memilih kekuatan peluru berdasarkan jarak dan kondisi energi. Ketika musuh dekat, bot menggunakan firepower tinggi untuk menghasilkan damage besar. Ketika musuh berada jauh, bot menggunakan firepower lebih kecil agar energi tidak cepat habis. Strategi ini cukup efektif ketika musuh memiliki pola gerak yang mudah diprediksi. Akan tetapi, ketika musuh bergerak secara acak, prediksi posisi target menjadi kurang akurat.

Dari hasil pengujian, dapat dilihat bahwa strategi greedy tidak selalu menghasilkan solusi optimal. Hal ini terjadi karena greedy hanya mengambil keputusan terbaik berdasarkan kondisi saat ini, bukan memperhitungkan seluruh kemungkinan langkah di masa depan. Dalam kondisi tertentu, seperti ketika musuh bergerak sangat acak atau ketika bot berada dekat dinding, keputusan greedy yang terlihat baik pada satu turn belum tentu memberikan hasil terbaik pada turn berikutnya.

Meskipun demikian, strategi TANK_Q tetap cukup efektif karena mampu menggabungkan tiga kemampuan utama, yaitu menyerang, bertahan, dan menghindar. Bot dapat memilih target, menentukan arah gerak, dan mengatur kekuatan tembakan secara adaptif. Dengan demikian, strategi greedy yang digunakan pada TANK_Q dapat dikatakan berhasil meningkatkan performa bot dibandingkan strategi sederhana yang hanya bergerak maju-mundur dan menembak secara langsung.

BAB 5

KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan perancangan, implementasi, dan pengujian bot Robocode Tank Royale, dapat disimpulkan bahwa algoritma greedy dapat digunakan sebagai strategi pengambilan keputusan pada bot permainan. Algoritma greedy bekerja dengan memilih keputusan terbaik berdasarkan kondisi saat itu, sehingga cocok diterapkan pada Robocode Tank Royale yang membutuhkan respons cepat pada setiap turn pertandingan. Pada tugas besar ini, terdapat empat alternatif solusi greedy yang dirancang, yaitu TANK_Q, BotKill, BotViper, dan VELORA. Setiap bot memiliki heuristic yang berbeda. TANK_Q menggunakan pendekatan Greedy Target Scoring & Multi-Candidate Greedy Movement, BotKill menggunakan pendekatan pemilihan musuh terdekat, BotViper menggunakan reactive dodging dan linear predictive aiming, sedangkan VELORA menggunakan opportunistic firepower dan randomized evasion. Dari keempat alternatif tersebut, TANK_Q dipilih sebagai bot utama karena memiliki strategi yang lebih seimbang antara menyerang, bertahan, dan menghindari.

Strategi greedy pada TANK_Q diterapkan dalam tiga bagian utama. Pertama, pada pemilihan target, bot memberikan skor kepada musuh berdasarkan jarak, energi musuh, dan konsistensi target. Musuh dengan skor tertinggi dipilih sebagai target utama. Kedua, pada pergerakan, bot mengevaluasi beberapa kandidat arah gerak dan memilih arah dengan skor terbaik berdasarkan keamanan dari dinding, jarak ideal terhadap musuh, dan pola orbit. Ketiga, pada penembakan, bot menentukan kekuatan peluru berdasarkan jarak target dan kondisi energi. Hasil pengujian menunjukkan bahwa strategi greedy pada TANK_Q mampu membantu bot mengambil keputusan secara lebih adaptif dibandingkan bot sederhana yang hanya bergerak maju-mundur dan menembak langsung. Bot dapat memilih target secara lebih terarah, menghindari dinding, mengubah arah ketika terkena peluru, serta menggunakan firepower secara lebih efisien. Hal ini menunjukkan bahwa pendekatan greedy cukup efektif untuk meningkatkan performa bot dalam pertandingan. Meskipun demikian, strategi greedy tetap memiliki keterbatasan. Karena greedy hanya mengambil keputusan terbaik pada kondisi saat itu, keputusan yang dipilih belum tentu menjadi keputusan terbaik untuk kondisi berikutnya. Bot masih dapat mengalami kesulitan ketika musuh bergerak sangat acak, ketika prediksi tembakan tidak akurat, atau ketika bot berada terlalu dekat dengan dinding. Dengan demikian, strategi TANK_Q dapat dikatakan berhasil menerapkan algoritma greedy, tetapi masih memiliki ruang untuk dikembangkan agar lebih stabil dan kompetitif.

5.2 Saran

Berdasarkan hasil implementasi dan pengujian yang telah dilakukan, masih terdapat beberapa hal yang dapat dikembangkan untuk meningkatkan performa bot. Salah satunya adalah peningkatan akurasi prediksi tembakan agar bot lebih efektif dalam menghadapi lawan dengan pola gerak yang tidak menentu. Selain itu, mekanisme pergerakan bot di dekat dinding arena juga masih dapat diperbaiki agar bot tidak mudah terjebak pada area sempit. Pemilihan target juga dapat dikembangkan dengan mempertimbangkan lebih banyak faktor selain jarak dan energi lawan, sehingga keputusan yang diambil menjadi lebih

optimal. Di sisi lain, pola penghindaran serangan dapat dibuat lebih bervariasi agar pergerakan bot tidak mudah diprediksi oleh lawan. Terakhir, pengujian sebaiknya dilakukan dengan lebih banyak variasi bot lawan untuk melihat efektivitas strategi greedy dalam berbagai kondisi pertandingan. Dengan pengujian yang lebih beragam, kelebihan dan kekurangan strategi yang digunakan dapat dianalisis dengan lebih baik.

LAMPIRAN

Lampiran : Tautan Repository GitHub

https://github.com/roy124140024-ctrl/Tubes_TANK-Q

DAFTAR PUSTAKA

Robocode Tank Royale, 2026, <https://github.com/robocode-dev/tank-royale>.

Munir, Rinaldi. “Algoritma Greedy (Bagian 1).” *Informatika STEI ITB*, 2021, <https://informatika.stei.itb.ac.id/~rinaldi.munir/Stmik/2020-2021/Algoritma-Greedy-%282021%29-Bag1.pdf>.

Munir, Rinaldi. “Kuliah 2 ITERA.” *Algortima Greedy Bagian 2*, 2025, https://kuliah2.itera.ac.id/pluginfile.php/70145/mod_resource/content/1/03a%20-%20Algoritma%20Greedy%20-%28Bagian%202%29.pdf.